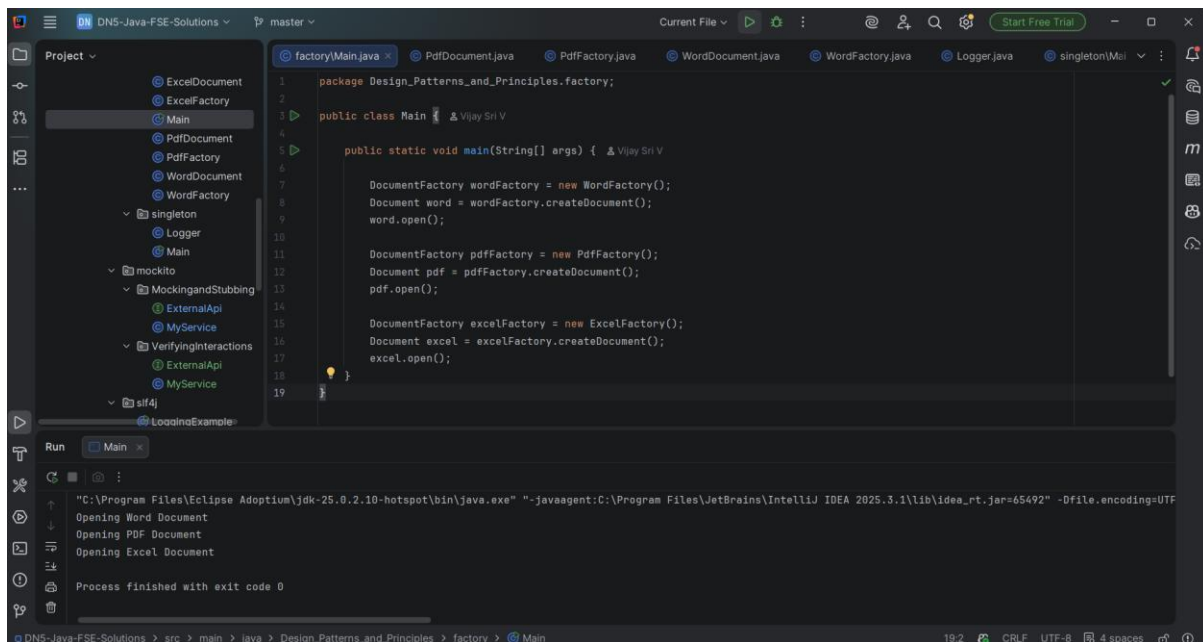


Design Patterns and principles

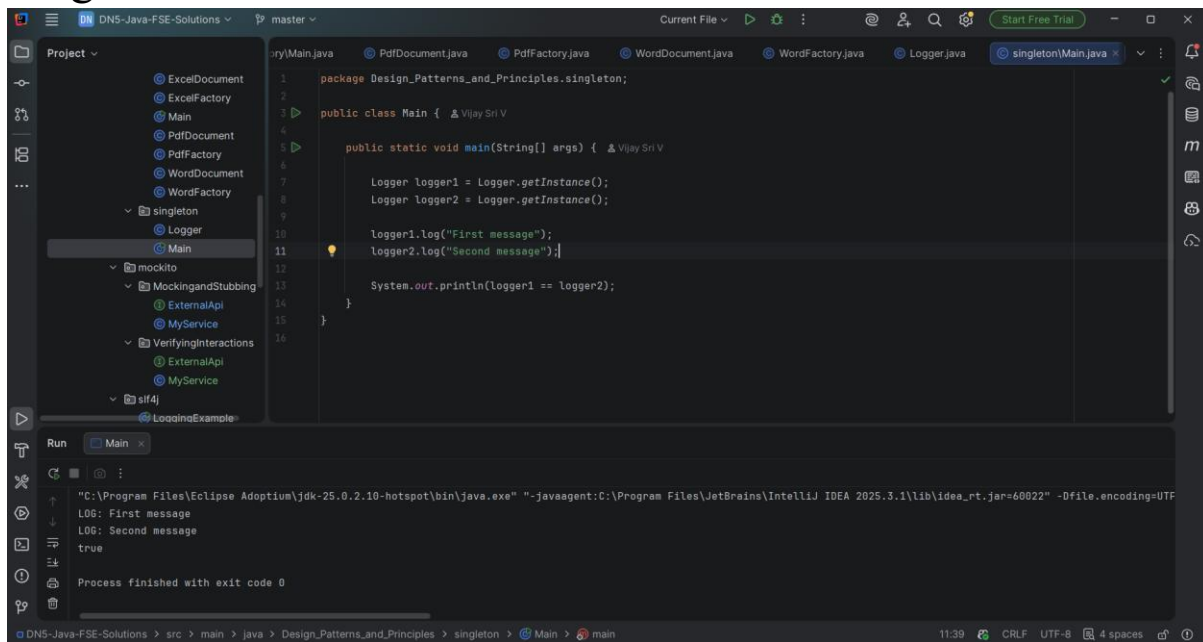
Factory method



The screenshot shows the IntelliJ IDEA interface with a project named "DNS-Java-FSE-Solutions". The project structure on the left includes a "factory" package with classes like ExcelDocument, ExcelFactory, Main, PdfDocument, PdfFactory, WordDocument, WordFactory, Logger, and singleton. The main editor displays the code for "factory/Main.java". The code defines a "Main" class with a "main" method that creates instances of "WordFactory", "PdfFactory", and "ExcelFactory" and uses them to create documents. The "Run" button is visible, and the output console shows the execution results.

```
1 package Design_Patterns_and_Principles.factory;
2
3 public class Main {
4     // Vijay Sri V
5
6     public static void main(String[] args) {
7         // Vijay Sri V
8
9         DocumentFactory wordFactory = new WordFactory();
10        Document word = wordFactory.createDocument();
11        word.open();
12
13        DocumentFactory pdfFactory = new PdfFactory();
14        Document pdf = pdfFactory.createDocument();
15        pdf.open();
16
17        DocumentFactory excelFactory = new ExcelFactory();
18        Document excel = excelFactory.createDocument();
19        excel.open();
20    }
21 }
```

Singleton method



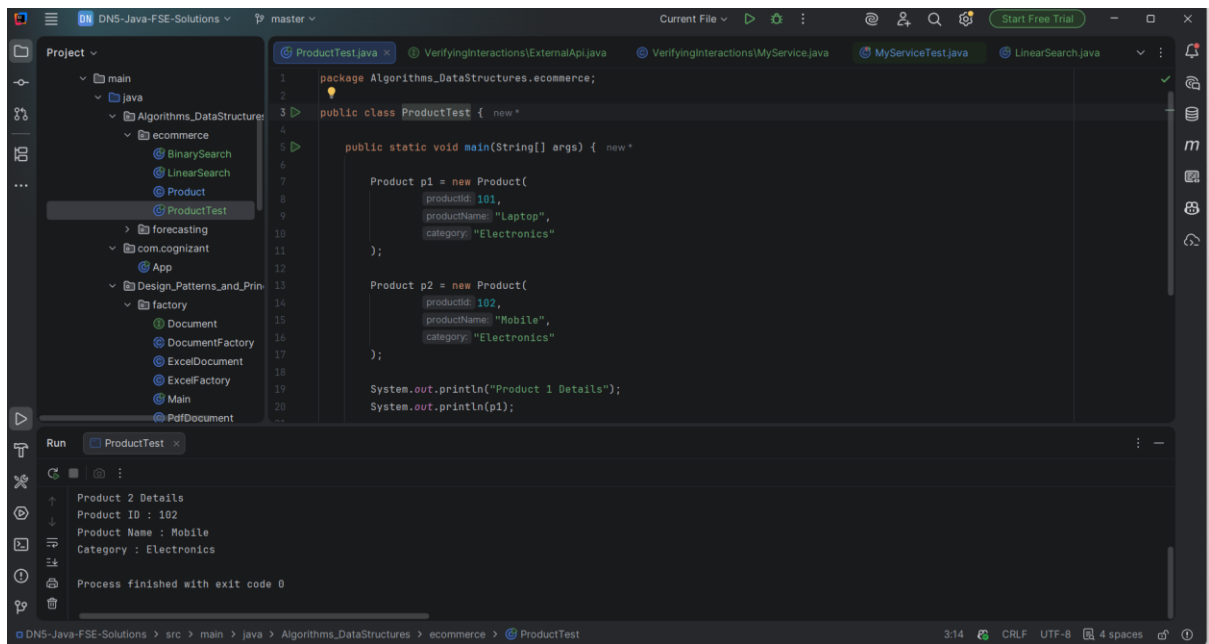
The screenshot shows the IntelliJ IDEA interface with the same project. The project structure on the left includes a "singleton" package with classes like Logger, Main, and singleton. The main editor displays the code for "singleton/Main.java". The code defines a "Main" class with a "main" method that uses the "Logger" class, which implements the Singleton pattern. The "Run" button is visible, and the output console shows the execution results, including the log messages.

```
1 package Design_Patterns_and_Principles.singleton;
2
3 public class Main {
4     // Vijay Sri V
5
6     public static void main(String[] args) {
7         // Vijay Sri V
8
9         Logger logger1 = Logger.getInstance();
10        Logger logger2 = Logger.getInstance();
11
12        logger1.log("First message");
13        logger2.log("Second message");
14
15        System.out.println(logger1 == logger2);
16    }
17 }
```

Algorithms and Data structures

Ecommerce

Product



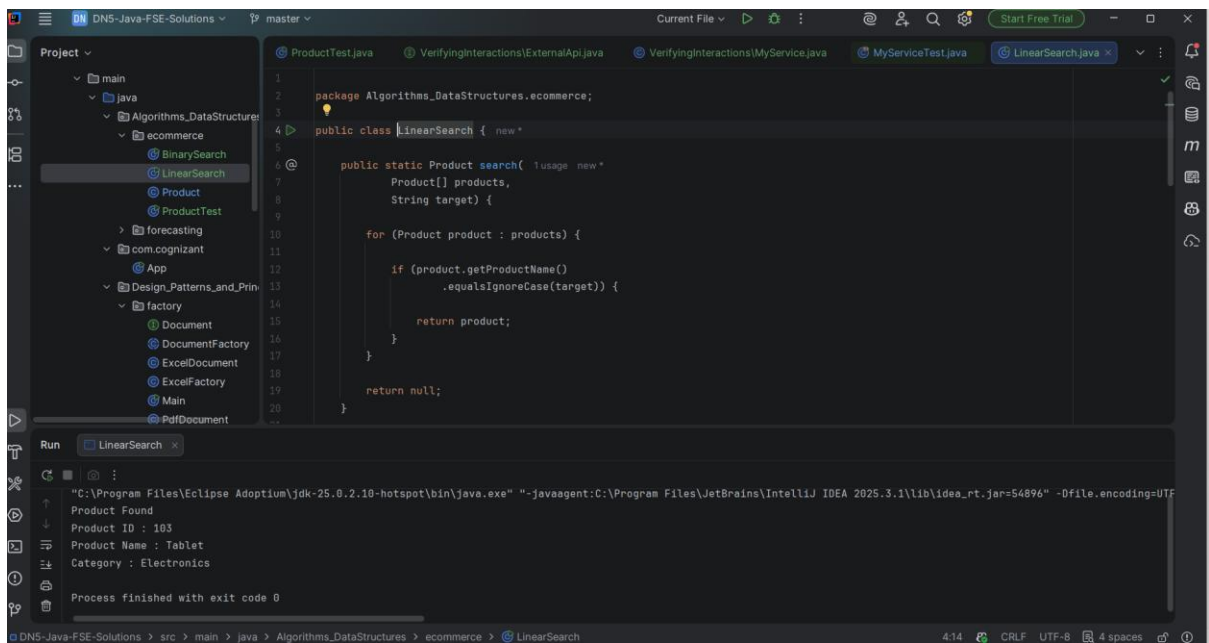
The screenshot shows the IntelliJ IDEA IDE with the `ProductTest.java` file open. The file is located in the package `Algorithms_DataStructures.ecommerce`. The code defines a `Product` class and a `ProductTest` class. The `Product` class has attributes `productId`, `productName`, and `category`. The `ProductTest` class has a `main` method that creates two `Product` objects, `p1` and `p2`, and prints their details.

```
1 package Algorithms_DataStructures.ecommerce;
2
3 public class ProductTest {
4     public static void main(String[] args) {
5
6         Product p1 = new Product(
7             productId: 101,
8             productName: "Laptop",
9             category: "Electronics"
10        );
11
12        Product p2 = new Product(
13            productId: 102,
14            productName: "Mobile",
15            category: "Electronics"
16        );
17
18        System.out.println("Product 1 Details");
19        System.out.println(p1);
20    }
21 }
```

The Run window shows the output of the `ProductTest` class:

```
Product 2 Details
Product ID : 102
Product Name : Mobile
Category : Electronics
Process finished with exit code 0
```

Linear Search



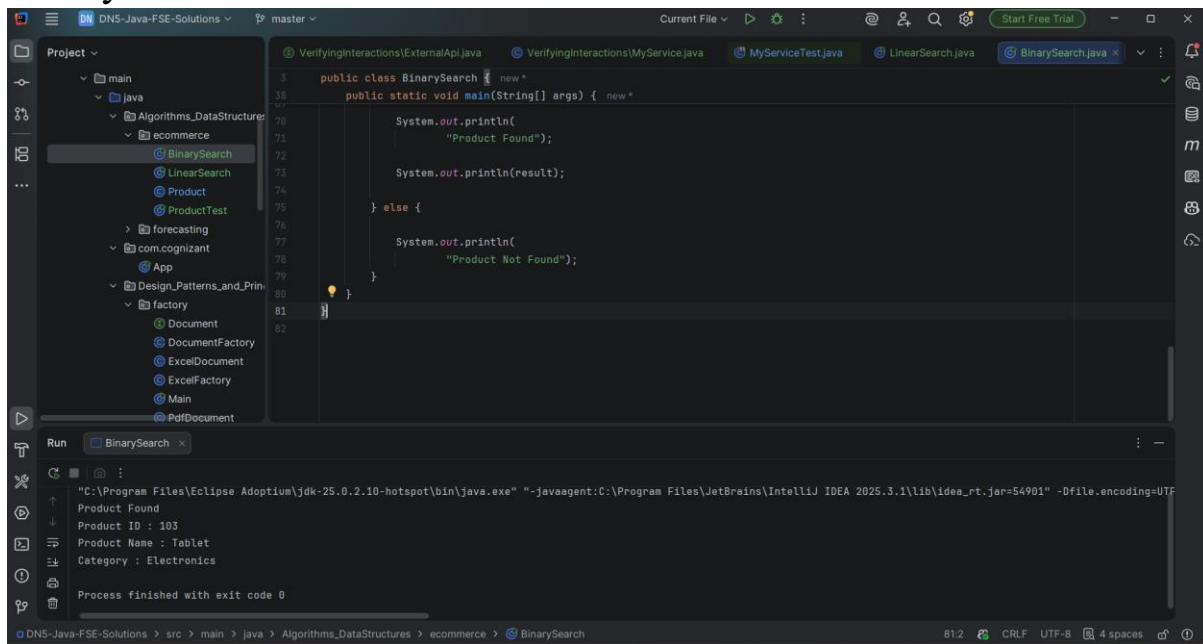
The screenshot shows the IntelliJ IDEA IDE with the `LinearSearch.java` file open. The file is located in the package `Algorithms_DataStructures.ecommerce`. The code defines a `LinearSearch` class with a `search` method that takes an array of `Product` objects and a target string, and returns the first `Product` object whose name matches the target (case-insensitive).

```
1 package Algorithms_DataStructures.ecommerce;
2
3 public class LinearSearch {
4     public static Product search(
5         Product[] products,
6         String target) {
7
8         for (Product product : products) {
9             if (product.getProductName().equalsIgnoreCase(target)) {
10                return product;
11            }
12        }
13        return null;
14    }
15 }
```

The Run window shows the output of the `LinearSearch` class:

```
Product Found
Product ID : 103
Product Name : Tablet
Category : Electronics
Process finished with exit code 0
```

Binary Search



The screenshot shows the IntelliJ IDEA IDE with the `BinarySearch.java` file open. The code implements a binary search algorithm. The `main` method takes an array of strings and a target string as input. It calls the `search` method to find the target. The output shows the product found and its details.

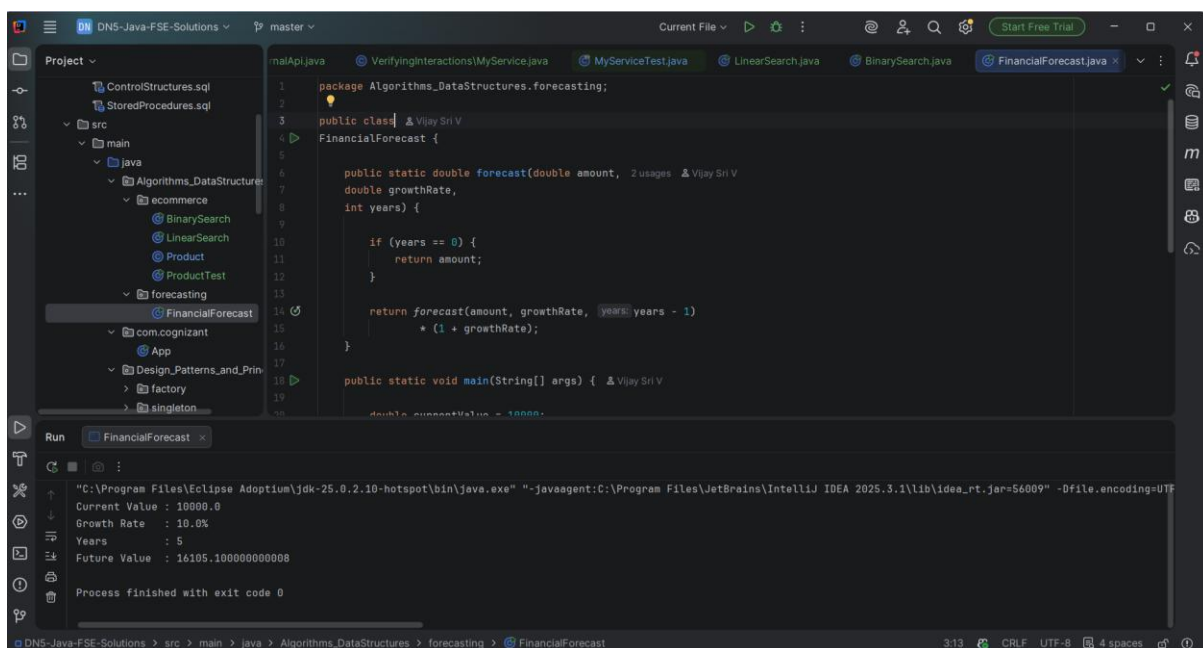
```
public class BinarySearch {  
    public static void main(String[] args) {  
        String[] products = {"Product A", "Product B", "Product C", "Product D", "Product E"};  
        String target = "Product B";  
        int result = search(products, target);  
        if (result != -1) {  
            System.out.println("Product Found");  
            System.out.println(result);  
        } else {  
            System.out.println("Product Not Found");  
        }  
    }  
}
```

Run Output:

```
"C:\Program Files\Eclipse Adoptium\jdk-25.0.2.10-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.1\lib\idea_rt.jar=54901" -Dfile.encoding=UTF-8  
Product Found  
Product ID : 103  
Product Name : Tablet  
Category : Electronics  
Process finished with exit code 0
```

Forecasting

Financial forecast



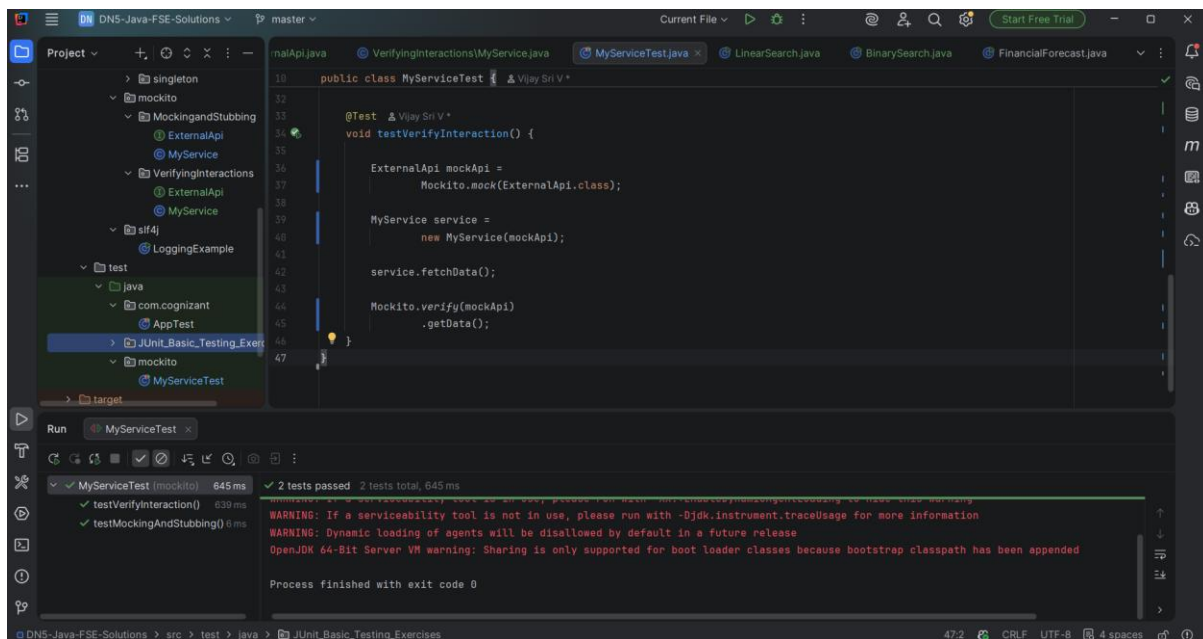
The screenshot shows the IntelliJ IDEA IDE with the `FinancialForecast.java` file open. The code implements a financial forecasting algorithm. The `main` method takes the current value, growth rate, and years as input. It calls the `forecast` method to calculate the future value. The output shows the current value, growth rate, years, and the calculated future value.

```
package Algorithms_DataStructures.forecasting;  
  
public class FinancialForecast {  
    public static double forecast(double amount, double growthRate, int years) {  
        if (years == 0) {  
            return amount;  
        }  
        return forecast(amount, growthRate, years - 1) * (1 + growthRate);  
    }  
  
    public static void main(String[] args) {  
        double currentValue = 10000.0;  
        double growthRate = 0.1;  
        int years = 5;  
        double futureValue = forecast(currentValue, growthRate, years);  
        System.out.println("Future Value : " + futureValue);  
    }  
}
```

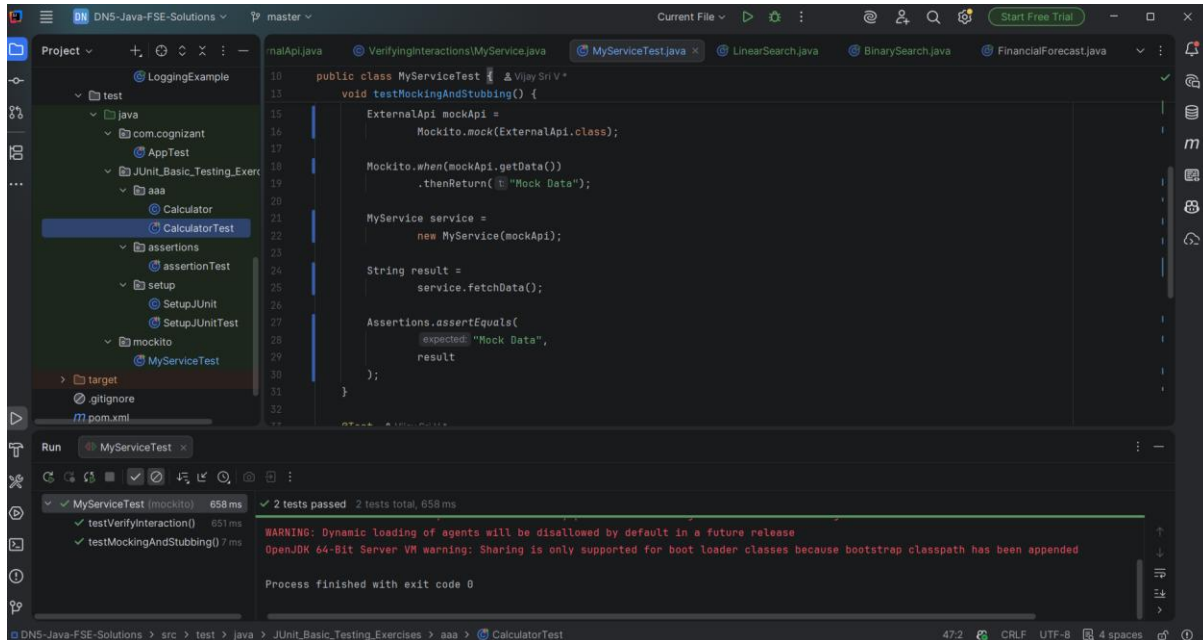
Run Output:

```
"C:\Program Files\Eclipse Adoptium\jdk-25.0.2.10-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.1\lib\idea_rt.jar=56009" -Dfile.encoding=UTF-8  
Current Value : 10000.0  
Growth Rate : 10.0%  
Years : 5  
Future Value : 16105.100000000008  
Process finished with exit code 0
```

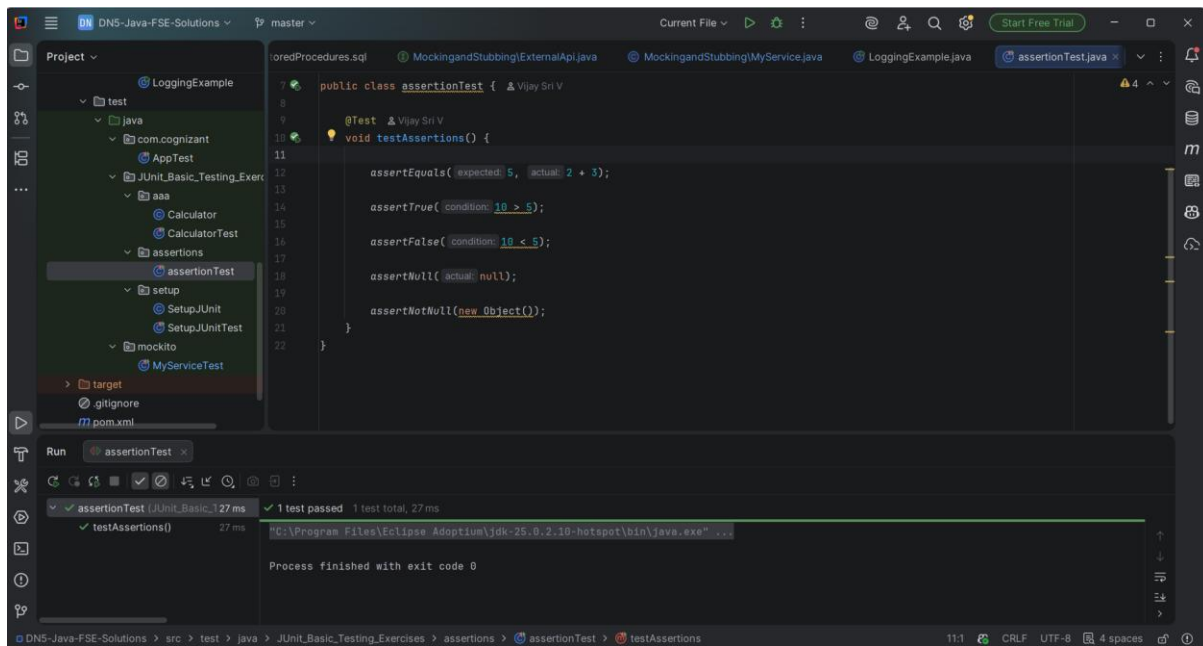
Mockito



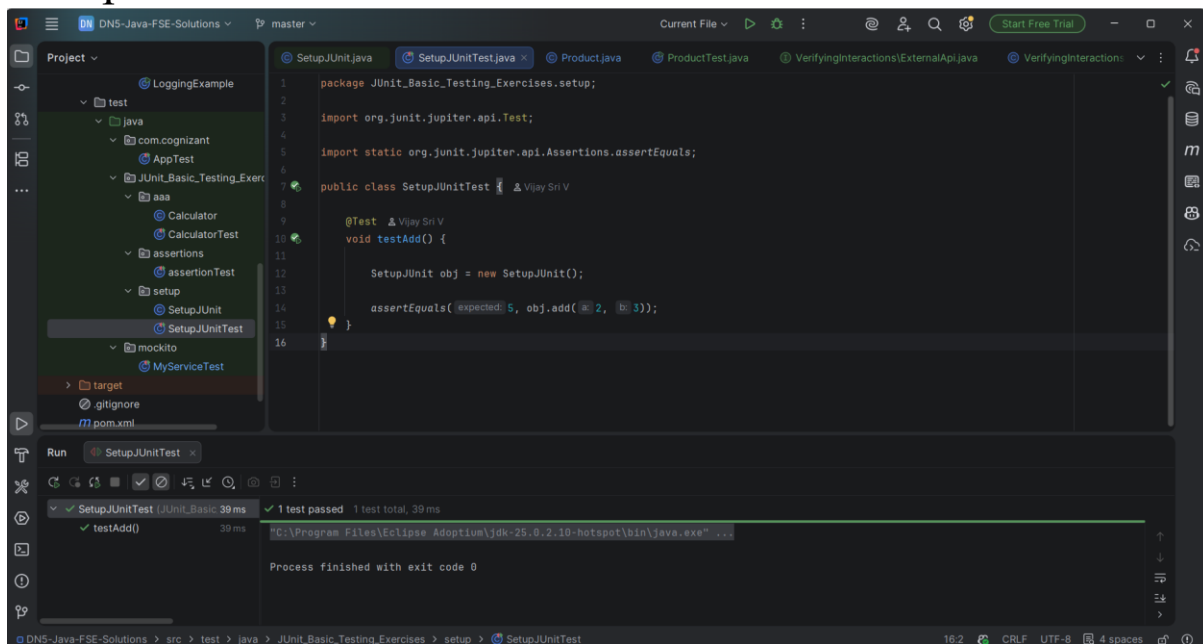
Junit AAA



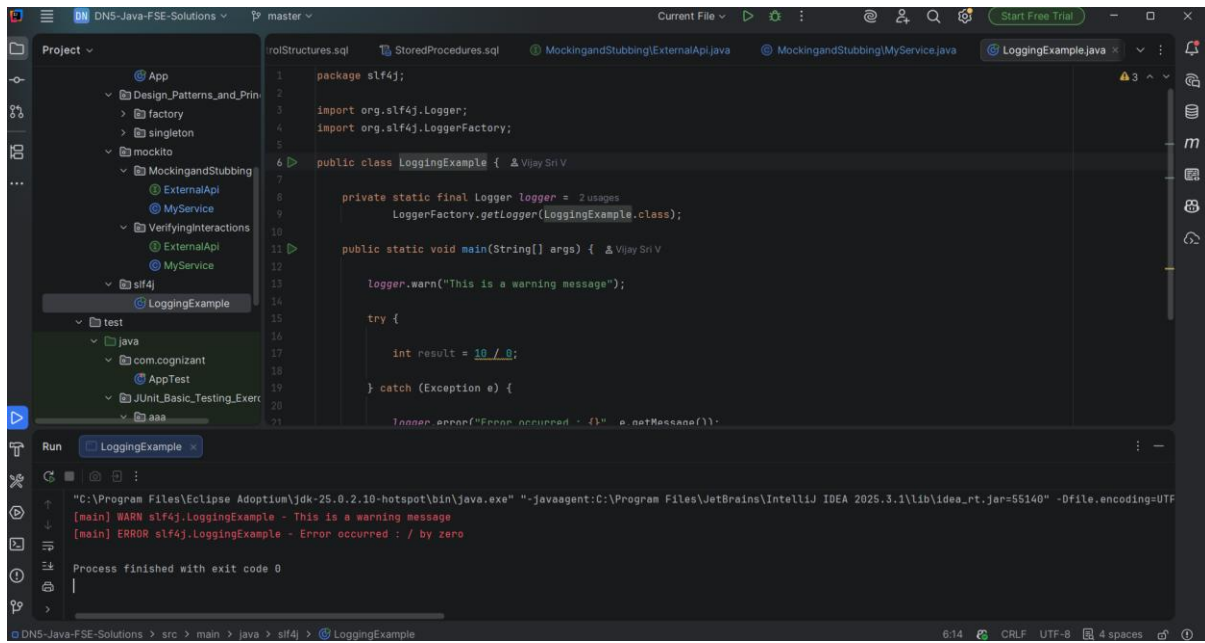
Assertions



SetUp Junit



SLF4J



The screenshot displays an IDE window for a project named "DNS-Java-FSE-Solutions". The left sidebar shows a project tree with a package structure including "slf4j" and "LoggingExample". The main editor shows the source code for "LoggingExample.java". The code defines a package, imports SLF4J classes, and implements a class with a logger and a main method that demonstrates logging and an exception.

```
1 package slf4j;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5
6 public class LoggingExample {
7
8     private static final Logger logger =
9         LoggerFactory.getLogger(LoggingExample.class);
10
11     public static void main(String[] args) {
12
13         logger.warn("This is a warning message");
14
15         try {
16
17             int result = 10 / 0;
18
19         } catch (Exception e) {
20
21             logger.error("Error occurred : {}", e.getMessage());
22         }
23     }
24 }
```

The bottom panel shows the output of the program execution:

```
"C:\Program Files\Eclipse Adoptium\jdk-25.0.2-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.1\lib\idea_rt.jar=55140" -Dfile.encoding=UTF
[main] WARN slf4j.LoggingExample - This is a warning message
[main] ERROR slf4j.LoggingExample - Error occurred : / by zero
Process finished with exit code 0
```